

Technical Assignment

Document Collection Chatbot

Duration

Complete the task within **one week**.

1. Objective

Develop a web application where users can organize PDF documents into collections and ask questions based on a selected collection.

The chatbot must answer only from documents inside the selected collection. When the answer is unavailable, it must clearly inform the user.

2. Required Technology Stack

Use:

- **Backend:** FastAPI
- **Frontend:** Next.js
- **Database:** PostgreSQL
- **Deployment:** Docker and Docker Compose

The developer may choose suitable libraries and implementation approaches.

3. User Authentication

Users must be able to:

- Register
- Log in
- Log out
- Access only their own data

Passwords must be stored securely.

Social login, password reset, email verification, user roles, and admin users are not required.

4. Document Collections

Users must be able to:

- Create a collection
- View their collections
- Rename a collection
- Delete a collection
- Select a collection for document upload and chat

Each collection should display:

- Collection name
- Creation date
- Number of documents

Users must only be able to access their own collections.

Deleting a collection must also remove its related documents and processed data.

5. Document Management

Users must be able to:

- Upload multiple PDF documents into a selected collection
- View documents inside a collection
- View document processing status
- Delete documents
- Retry failed document processing

Each document should display:

- Filename
- Upload date
- Status
- Error message, when applicable

Only PDF documents are required.

Deleted documents must no longer be used by the chatbot.

6. Document Processing

The system must:

- Extract text from uploaded PDFs
- Prepare the text for retrieval
- Generate embeddings using a local embedding model
- Store the processed document data
- Associate processed data with the correct user and collection
- Mark documents as ready or failed

External embedding APIs must not be used.

OCR for scanned PDFs is not required.

The developer may choose the extraction, chunking, embedding, and storage approach.

7. RAG Chatbot

Users must be able to select a collection and ask questions based only on documents inside that collection.

The chatbot must:

- Search only the selected collection
- Search only documents belonging to the authenticated user
- Retrieve relevant content
- Generate answers using only the retrieved content
- Display the source document name
- Display the page number when available
- Return a fallback response when the answer is unavailable

Fallback response:

I could not find this information in the selected collection.

The chatbot must never use documents from another collection or another user.

8. AI Model Usage

Use:

- A **local embedding model** for document and question embeddings
- **deepseek-v4-flash** as the only API-based language model

LITELLM_BASE_URL=https://lite-llm.datafabdevelopment.com/v1

LITELLM_MODEL=deepseek-v4-flash

LITELLM_API_KEY=**sk-RIMeUq5cRCdvISJI8grSHQ**

The API key must be stored using environment variables and must not be committed to Git.

9. User Interface

The application must include:

- Registration page
- Login page
- Collection management section
- Collection selection
- Document upload section
- Document list
- Document status display
- Chat interface
- Loading states
- Error messages
- Logout option

The interface can be simple and functional.

10. Optional Chat History

Chat history is optional.

As an additional feature, the application may support:

- Previous questions and answers
- Multiple conversations
- Starting a new conversation
- Deleting conversations

Any stored chat history must belong to the authenticated user and the selected collection.

11. Docker Deployment

The complete application must run using Docker Compose.

The setup must include:

- FastAPI backend
- Next.js frontend
- PostgreSQL database
- Any other required local service

The application should run using:

`docker compose up --build`

The README must explain the setup, environment variables, ports, and startup process.

12. Security Requirements

The application must ensure that:

- Users access only their own collections
 - Users access only their own documents
 - Users query only the selected collection
 - Passwords are stored securely
 - Protected features require authentication
 - API keys are not stored in source code
 - Invalid files are rejected
 - Internal errors are not exposed to users
-

13. Testing

Test the main flows:

- Registration and login
- Invalid login
- Collection creation
- Collection update and deletion
- PDF upload into a collection
- Invalid file upload
- Document processing
- Document deletion
- Failed document retry
- Asking a question from a selected collection
- Successful chatbot answer
- Fallback response
- Collection data isolation
- User data isolation
- Docker startup

Automated backend tests should be included for the main flows.

14. Deliverables

Submit:

- FastAPI backend
- Next.js frontend
- PostgreSQL configuration
- Collection management implementation
- RAG implementation
- Local embedding model integration
- Dockerfiles
- Docker Compose configuration
- Database setup or migration files
- Automated backend tests
- `.env.example`
- README

- Sample PDF files
-

15. AI Tool Usage

AI-assisted development tools are allowed.

However, the developer must have a strong understanding of:

- The application architecture
- The complete codebase
- Authentication
- Database design
- Collection and document management
- Document processing
- Embeddings and retrieval
- RAG flow
- Docker setup
- Testing and error handling

The developer must be able to explain, modify, debug, and extend the submitted code.

Code that cannot be explained or modified may be considered incomplete.

16. Out of Scope

The following are not required:

- Admin dashboard
- User roles
- Password reset
- Email verification
- Social login
- OCR

- Non-PDF documents
 - Document sharing
 - Voice features
 - Advanced analytics
 - Advanced frontend design
 - Billing or subscriptions
-

17. Completion Criteria

The task is complete when:

1. Users can register and log in.
2. Users can create and manage collections.
3. Users can upload multiple PDFs into a collection.
4. Documents are processed using a local embedding model.
5. Users can select a collection and ask questions.
6. Only documents inside the selected collection are searched.
7. Answers are generated using `deepseek-v4-flash`.
8. Source document information is shown.
9. A fallback response is returned when needed.
10. Users cannot access another user's collections or documents.
11. Collections and documents can be deleted.
12. Failed documents can be retried.
13. The full application runs using Docker Compose.